

Ultrasonic Water-Level Sensor & Controller

Arduino Uno · HC-SR04 · L298N · 16×2 I²C LCD — main @ 8f3b886 — documented 2026-10-01 — MIT

Confidence labels: VERIFIED (read in source) · DOCUMENTED (README/deck/diagram) · INFERRED (analysis) · UNKNOWN. Machine-readable source of truth: CODEMAP.md.

1. Project Identity

Field	Value
Name	Ultrasonic Water-Level Sensor & Controller
Purpose	Keep a water tank between 10 % and 90 % full by measuring the level ultrasonically and switching a pump.
Problem solved	Tanks overflow (or run dry) because nobody watches them.
Type	Embedded firmware + hardware prototype (university mini-project, Basic Electronics BBEE203, AIMT Bhatkal)
Status	Hardware built and demonstrated (DOCUMENTED). The original controller firmware was lost. src/water_level_controller.ino is a rewrite from the design, logic-tested only and not re-run on hardware .
Languages	Arduino C++ (.ino), C++ (PC test)
Hardware	Arduino Uno, HC-SR04, 16×2 I ² C LCD (0x27), L298N, submersible DC pump, push-button, rocker switch, 12 V adapter
Libraries	NewPing, Wire, LiquidCrystal_I2C
Repo	github.com/Sallu-k/Ultrasonic-Water-Level-Sensor-and-Controller, branch main, HEAD 8f3b886
License	MIT, © 2025 Salsabeel Kobattey
Documented	2026-10-01

2. Project Overview

An HC-SR04 ultrasonic sensor is mounted in the tank lid, looking down. It measures the time of flight to the water surface. An Arduino Uno converts the echo time into distance (cm), and the distance into a percentage full. The mapping is **inverted**: a larger distance means less water. Every 300 ms, a two-threshold hysteresis controller decides the pump state: ON below 10 %, OFF at or above 90 %, and the previous state is kept in between. The decision drives an L298N H-bridge that switches a 12 V submersible pump. A 16×2 I²C LCD shows the level, the pump state and a manual flag. A debounced push-button toggles a manual override, which starts the pump early but cannot defeat the 90 % cut-off.

Safety layers:

- the pump is forced OFF at boot;
- 5 consecutive no-echo readings set a sensor fault and force the pump OFF;
- the override is cancelled when the tank is full;
- a plumbed overflow route keeps spilled water away from the electronics (DOCUMENTED, physical).
- Users:** a household or tank owner, who reads the LCD and can press the override button.
- Inputs:** the ultrasonic echo and the button.
- Outputs:** the pump drive (IN1/IN2/ENA), the LCD, and a serial log at 9600 baud.
- External services / network / database:** none.

3. Directory Map

```
/
├── README.md                overview, pin map, status
├── LICENSE                  MIT
├── CODEMAP.md / CODEMAP.pdf  this learning map
├── src/
│   ├── water_level_controller.ino  ENTRY POINT – controller (rewrite)
│   ├── water_level_measure_ORIGINAL.ino  original sensing-only sketch (2024), kept as-is
│   └── water_level_measure.ino      byte-identical copy of ORIGINAL
├── test/
│   ├── logic_test.cpp           PC-side tests of the control logic (16 checks)
│   └── README.md
├── docs/
│   ├── images/                 system_block_diagram.(png|svg), hysteresis_cycle.svg, final_build.jpeg
│   ├── report/                 PROJECT_REPORT.(md|pdf), seminar_deck.pptx
│   └── video/                   demo_presentation.mp4
```

- `src/` — Arduino sketches. Only `water_level_controller.ino` is the current firmware.
- `test/` — a host-compiled copy of the pure logic functions, plus checks.
- `docs/` — diagrams, a photo, the detailed report, the original seminar deck and the demo video.
- No build system, manifest, CI or configuration files exist. Configuration lives in `const` values inside the sketch.

4. Component Map

Component: ultrasonic_sensing

- **Purpose:** measure the distance to the water surface.
- **Files:** `src/water_level_controller.ino` (`sonar`, part of `loop()`)
- **Inputs:** HC-SR04 echo pulse on D10. TRIG is on D12.
- **Outputs:** `distance` in whole cm (`unsigned int`); 0 means no echo.
- **Depends on:** `NewPing` (`ping_median`, `convert_cm`)
- **Used by:** `level_conversion`, `fault_safety`
- **Behaviour:** a 5-ping median (`PING_SAMPLES`), with a 200 cm time-out ceiling (`MAX_DISTANCE`).
- **Evidence:** VERIFIED in source.

Component: level_conversion

- **Purpose:** convert distance to percentage full.
- **Files:** `distanceToPercent(float)`
- **Inputs:** distance in cm, `SENSOR_TO_EMPTY = 14.0`, `SENSOR_TO_FULL = 3.0`
- **Outputs:** an integer from 0 to 100 %
- **Behaviour:** an inverted linear map, clamped to 0–100, rounded (+0.5). A span ≤ 0 returns 0.
- **Used by:** `control_law`, `lcd_display`
- **Evidence:** VERIFIED; tested.

Component: control_law

- **Purpose:** hysteresis pump decision.
- **Files:** `updatePump(int)`
- **Inputs:** `level %`, `sensorFailed`, `overrideOn`, `pumpRunning`
- **Outputs:** `pumpRunning` (and it may clear `overrideOn`)
- **Priority order:** `fault` → OFF; ≥ 90 → OFF and cancel override; `override` → ON; < 10 → ON; otherwise hold.
- **Configuration:** `PUMP_ON_BELOW = 10`, `PUMP_OFF_ABOVE = 90`
- **Used by:** `main_loop` → `pump_actuation`
- **Evidence:** VERIFIED; tested.

Component: manual_override

- **Purpose:** let the user start the pump early.

- **Files:** `overridePressed()`, the toggle in `loop()`
- **Inputs:** button on D2 (`INPUT_PULLUP`, LOW = pressed)
- **Outputs:** toggles `overrideOn`
- **Behaviour:** 50 ms debounce (`DEBOUNCE_MS`); fires once per press. The 90 % cut-off beats it.
- **Evidence:** VERIFIED.

Component: fault_safety

- **Purpose:** fail safe on sensor loss and at boot.
- **Files:** `setup()` (pump OFF first); the `distance == 0` branch in `loop()`; the `sensorFailed` check in `updatePump()`
- **Behaviour:** `badReadings` counts consecutive zero readings. At `MAX_BAD_READINGS = 5` (about 1.5 s) it sets `sensorFailed`, turns the pump OFF, and shows "SENSOR FAULT" on the LCD and serial. The first good reading clears the fault.
- **Physical layer:** an overflow outlet routed away from the electronics (DOCUMENTED, README only).
- **Evidence:** VERIFIED (software).

Component: pump_actuation

- **Purpose:** drive the L298N.
- **Files:** `applyPumpOutput()`
- **Outputs:** IN1 (D7) = `pumpRunning`, IN2 (D8) = LOW, ENA (D6) = `pumpRunning`
- **Depends on:** a shared ground between the Arduino and the L298N (documented in the source header).
- **Evidence:** VERIFIED.

Component: lcd_display

- **Purpose:** show the level and the pump state.
- **Files:** `updateDisplay(int)`
- **Behaviour:** redraws only when something changes (static "shown" cache), never calls `lcd.clear()`, and pads lines to 16 characters. Line 0 shows `Level: NN%` or `SENSOR FAULT`. Line 1 shows `Pump: ON/OFF[MANUAL]`.
- **Depends on:** Wire, LiquidCrystal_I2C at 0x27 (A4/A5)
- **Evidence:** VERIFIED.

Component: main_loop

- **Purpose:** non-blocking scheduler and serial log.
- **Files:** `setup()`, `loop()`
- **Behaviour:** polls the button on every pass. Uses `millis()` gating at `SAMPLE_INTERVAL_MS = 300` to run `sense` → `convert` → `decide` → `actuate` → `display` → `log`. Serial at 9600 baud: `dist=..cm level=..% pump=ON|OFF [MANUAL]`.
- **Evidence:** VERIFIED.

Component: logic_tests

- **Purpose:** verify the decision logic on a PC.
- **Files:** `test/logic_test.cpp`, `test/README.md`
- **Behaviour:** **duplicates** `distanceToPercent` and `updatePump` without Arduino calls. Runs 16 checks. The exit code is the failure count.
- **Evidence:** VERIFIED. The copies match the sketch at HEAD.

Component: legacy_measure_sketch

- **Purpose:** historical record of the first sensing and display stage.
- **Files:** `src/water_level_measure_ORIGINAL.ino`, `src/water_level_measure.ino` (identical)
- **Behaviour:** `ping_cm()`. If $0 < d < 14$, it shows "Deep" and `12.50 - d`. Otherwise it does not update (stale display). Uses `delay(150)`. No pump logic.
- **Evidence:** VERIFIED. The README says it was kept unmodified on purpose.

Component: power_and_wiring

- **Purpose:** physical platform.
- **Facts:**
- 12 V adapter → rocker switch → L298N motor supply and Arduino supply (DOCUMENTED in the block diagram).

- The sensor and LCD run from the Arduino's 5 V (source header).
- The tank is about 5 L (diagram label).
- **Evidence:** DOCUMENTED. Exact Arduino power pin (VIN, barrel jack, or the L298N 5 V output) is NOT VERIFIED.

5. File Responsibility Map

File	Component	Responsibility	Important symbols
src/water_level_controller.ino	all firmware components	Current controller firmware	distanceToPercent, updatePump, applyPumpOutput, overridePressed, updateDisplay, setup, loop, pumpRunning, overrideOn, sensorFailed, badReadings, SENSOR_TO_EMPTY/FULL, PUMP_ON_BELOW/OFF_ABOVE
src/water_level_measure_ORIGINAL.ino	legacy_measure_sketch	Original sensing + LCD stage	loop (ping_cm, 12.50 - distance)
src/water_level_measure.ino	legacy_measure_sketch	Duplicate of ORIGINAL; purpose UNKNOWN	—
test/logic_test.cpp	logic_tests	Host tests of conversion and control	chk, main, copies of the 2 functions
README.md	—	Overview, pin map, status, test command	—
docs/images/system_block_diagram.svg	power_and_wiring	System block diagram (power, signals, water)	—
docs/report/PROJECT_REPORT.md	—	Long-form engineering report	—
docs/report/seminar_deck.pptx	—	Original 2024 seminar: motivation, objectives, references	—

6. Relationship Map

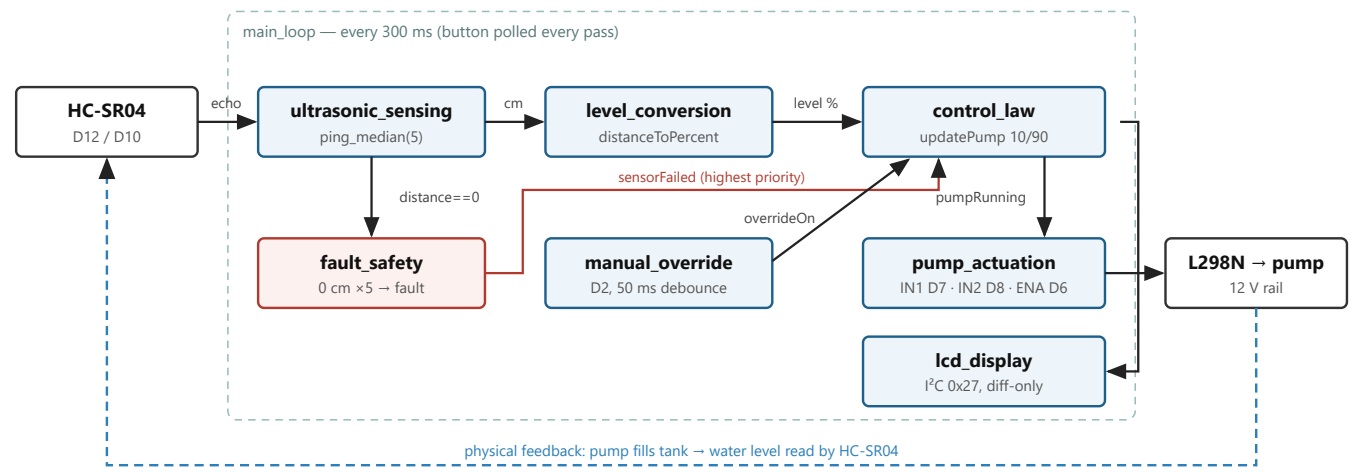


Figure 1 — Component data flow. Boxes use CodeMap component IDs; every edge is verified in `src/water_level_controller.ino`. Red = safety path; blue dashed = physical loop.

```

HC-SR04 → ultrasonic_sensing      : echo pulse (D10), triggered by D12
ultrasonic_sensing → fault_safety : distance==0 → badReadings++
ultrasonic_sensing → level_conversion : distance cm (non-zero)
level_conversion → control_law     : level %
manual_override → control_law      : overrideOn
fault_safety → control_law          : sensorFailed (highest priority)
control_law → manual_override      : clears overrideOn at ≥90 %
control_law → pump_actuation        : pumpRunning
pump_actuation → L298N → pump       : IN1/IN2/ENA, 12 V rail
control_law/level → lcd_display     : level, pump, override, fault
main_loop → all                     : schedules every 300 ms; button every pass
logic_tests → level_conversion, control_law : duplicated code (no shared header)
pump → tank → HC-SR04              : physical feedback loop

```

7. Main Workflows

W1 Boot

1. Set pin modes (button `INPUT_PULLUP`).
2. Set `pumpRunning=false` and call `applyPumpOutput()`. This happens **before** anything else.
3. Initialise the LCD with a splash ("Water Level Ctrl / Starting...").
4. Start serial at 9600 baud and print "ready".
5. `delay(1000)`.

W2 Control cycle (every 300 ms)

1. Poll the button; a debounced press toggles `overrideOn`.
2. If less than 300 ms have passed since the last sample, return.
3. `ping_median(5) → convert_cm()`.
4. If `distance == 0`, go to W3 and return.
5. Reset `badReadings` and clear `sensorFailed`.
6. `level = distanceToPercent(distance)`.
7. `updatePump(level) → applyPumpOutput() → updateDisplay(level)`.
8. Log the cycle to serial.

W3 Sensor fault

1. Each zero reading increments `badReadings` (capped at 5).
2. At 5 readings (and not already failed): set `sensorFailed`, pump OFF, LCD shows "SENSOR FAULT", print a serial message.
3. Fault recovery: the next non-zero reading clears the fault in W2 step 5.

W4 Manual top-up

1. The user presses the button while the level is between 10 and 89 %. The override turns on and the pump turns on.
2. The level reaches 90 % or more. The pump turns off and the override clears automatically.

W5 Calibration (DOCUMENTED in the report, manual)

1. Read `dist=` on serial with the tank empty → `SENSOR_TO_EMPTY`.
2. Read `dist=` with the tank full → `SENSOR_TO_FULL` (the smaller value).
3. Re-flash.

8. Data Flow

```

INPUT      echo time (µs) ×5 — button level (D2)
PROCESSING median → cm (int, /57) → % = (EMPTY-d)/(EMPTY-FULL)·100, clamp, round
DECISION   fault? → OFF | ≥90 → OFF + cancel override | override → ON | <10 → ON | hold
OUTPUT     D7/D8/D6 → L298N → 12 V pump · I²C LCD (2×16 chars) · Serial 9600 log

```

9. Important Logic

Logic	What it does	Why it exists	Where	Assumptions
Hysteresis (10/90)	Holds pump state inside the band	Prevents chatter from surface ripple	<code>updatePump</code>	<code>pumpRunning</code> persists between calls
Priority ordering	<code>fault > full cut-off > override > low threshold</code>	Override cannot overfill; fault always wins	<code>updatePump</code>	—
Inverted mapping + clamp	Larger distance means a lower %; clamped to 0–100	Sensor looks down; stray echoes	<code>distanceToPercent</code>	EMPTY > FULL; constants are calibrated
Median filter	Middle value of 5 pings	Rejects outliers without averaging lag	<code>loop</code> (<code>ping_median</code>)	NewPing behaviour
Zero = no echo	Never treats 0 as a distance	Avoids a false "full"/"empty"	<code>loop</code>	NewPing returns 0 on time-out
Dropout counter	5 consecutive zeros → fault	Tolerates single dropouts	<code>loop</code>	$300\text{ ms} \times 5 \approx 1.5\text{ s}$
Safe boot	Pump OFF before init	Pin state undefined at power-up	<code>setup</code>	—
millis scheduling	Non-blocking 300 ms period	Button stays responsive	<code>loop</code>	Unsigned rollover-safe subtraction
Debounce	Accepts state after 50 ms stable	Contact bounce	<code>overridePressed</code>	Active-LOW button
Diff-only LCD	Redraws on change, fixed-width padding	No flicker; no leftover digits	<code>updateDisplay</code>	16-char lines

10. Configuration

All configuration is compile-time `const` in `src/water_level_controller.ino`. There are no secrets and no environment variables.

Setting	Purpose	Value
<code>TRIGGER_PIN</code> / <code>ECHO_PIN</code>	HC-SR04	12 / 10
<code>PUMP_IN1</code> / <code>PUMP_IN2</code> / <code>PUMP_ENA</code>	L298N	7 / 8 / 6
<code>OVERRIDE_BTN</code>	Button	2
<code>SENSOR_TO_EMPTY</code> / <code>SENSOR_TO_FULL</code>	Tank geometry; must be calibrated	14.0 / 3.0 cm (inferred from the 2024 sketch)
<code>PUMP_ON_BELOW</code> / <code>PUMP_OFF_ABOVE</code>	Thresholds	10 / 90 %
<code>SAMPLE_INTERVAL_MS</code>	Sample period	300
<code>PING_SAMPLES</code>	Median depth	5
<code>MAX_BAD_READINGS</code>	Fault threshold	5
<code>DEBOUNCE_MS</code>	Debounce	50
<code>MAX_DISTANCE</code>	NewPing ceiling	200 cm
LCD address / size	<code>LiquidCrystal_I2C lcd(0x27, 16, 2)</code>	0x27, 16×2

11. Hardware Map

Hardware	Interface	Connected to	Purpose	Confidence
Arduino Uno	—	—	MCU	VERIFIED
HC-SR04	GPIO	TRIG D12, ECHO D10, 5 V, GND	Distance	VERIFIED (source)
16×2 LCD + I ² C backpack	I ² C 0x27	SDA A4, SCL A5, 5 V, GND	Display	VERIFIED

Hardware	Interface	Connected to	Purpose	Confidence
L298N	GPIO	IN1 D7, IN2 D8, ENA D6, common GND	Pump driver	VERIFIED
Submersible DC pump	12 V via L298N out	L298N channel A	Fill tank	DOCUMENTED
Push-button	GPIO, internal pull-up	D2 → GND	Override	VERIFIED
Rocker switch	Power	12 V adapter → system	Isolation	DOCUMENTED
12 V DC adapter	Power	Switch → L298N and Arduino	Supply	DOCUMENTED; Arduino power pin NOT VERIFIED
Overflow outlet	Plumbing	Tank → away from electronics	Physical fail-safe	DOCUMENTED

12. Software Stack

- **Language:** Arduino C++ (firmware), C++ (tests)
- **Framework:** Arduino core (AVR, ATmega328P)
- **Libraries:** NewPing, Wire, LiquidCrystal_I2C (versions UNKNOWN; no manifest)
- **Build tools:** Arduino IDE (implied by `.ino`); `g++` for the tests
- **Runtime:** bare-metal Uno; tests run on any OS with `g++`
- **Database / external services / OS:** none

13. Important Commands

Purpose	Command	Source
Run logic tests	<code>cd test && g++ -Wall -O1 -o logic_test logic_test.cpp && ./logic_test</code>	README, test/README
Flash firmware	Arduino IDE: open <code>src/water_level_controller.ino</code> → board "Arduino Uno" → Upload	INFERRED (no CLI script)
Monitor	Serial Monitor at 9600 baud	<code>Serial.begin(9600)</code>

There is no install script, build script, deployment step or CI.

14. Known Problems / Failure Modes

#	Problem	Cause	Effect	File	Fix / workaround	Label
P1	Original controller lost	—	The current firmware is a rewrite	README	Re-test on hardware	DOCUMENTED
P2	Rewrite not hardware-tested	—	Only the logic is verified	README	Flash, calibrate, re-demo	DOCUMENTED
P3	Legacy stale display	LCD updates only when <code>d < 14 cm</code>	Shows an old value indefinitely	<code>*_ORIGINAL.ino</code>	Kept on purpose; fixed in the rewrite	DOCUMENTED
P4	Geometry defaults are guesses	Inferred from the 2024 sketch	Wrong % on a real tank	<code>controller</code>	Calibrate (W5)	DOCUMENTED
P5	1 cm resolution	<code>convert_cm()</code> returns an integer	Only 12 levels on an 11 cm span (about 9 % steps); actual switching at 9 % / 91 %	<code>controller loop</code>	Use <code>us / 57.0</code> as a float	INFERRED
P6	LCD line 1 overflow	"Pump: ON [MANUAL]" is 17 characters	Shows <code>...[MANUAL</code> (truncated, safe)	<code>updateDisplay</code>	Shorter tag	INFERRED

#	Problem	Cause	Effect	File	Fix / workaround	Label
P7	Override persists through a fault	Fault path does not clear <code>overrideOn</code>	Pump restarts on recovery if level < 90 %	<code>loop</code> , <code>updatePump</code>	Clear on fault (still bounded by 90 %)	INFERRED
P8	Button can miss very short taps	<code>ping_median(5)</code> blocks up to about 150 ms	Rare missed press	<code>loop</code>	<code>ping_timer()</code> / interrupt	INFERRED
P9	Near the sensor blind zone	FULL = 3 cm vs HC-SR04 minimum of about 2 cm	Least reliable echoes near full	constants	Mount higher	INFERRED
P10	No dry-run protection	Only the tank level is sensed	Pump can run dry if its source empties	—	Float switch or time-out	INFERRED
P11	Motor driver "does nothing"	No common ground	Pump never runs	header comment	Share GND	DOCUMENTED (source comment)

15. Testing

- **Framework:** none; a hand-rolled `chk()` in `test/logic_test.cpp`. The exit code is the failure count.
- **Command:** see §13.
- **Tested (16 checks):** conversion endpoints, midpoint and clamping (6); fill/drain switching points at 0, 90 and 9 %, with 3 transitions per cycle (4); no chatter on 47–53 % jitter, with state held (2); override starts the pump, the 90 % cut-off beats it, and it auto-cancels (3); a fault stops the pump even with override on (1).
- **Status:** README says all pass. On 2026-10-01 a line-for-line Python port of the 16 checks passed 16/16. `g++` was unavailable, so the C++ file itself was not compiled.
- **Untested:** NewPing and the sensor, the zero-reading/fault counter in `loop()`, debouncing, the LCD, the L298N, timing, and the legacy sketch.
- **Risk:** the tested functions are copies; there is no shared header.

16. Important Files To Learn First

1. `src/water_level_controller.ino` — the whole system; heavily commented design rationale.
2. `README.md` — purpose, control law, fail-safe philosophy, status.
3. `docs/images/system_block_diagram.svg` — physical architecture and power.
4. `test/logic_test.cpp` — executable specification of the control behaviour.
5. `src/water_level_measure_ORIGINAL.ino` — the "before", for contrast with the rewrite.
6. `docs/report/PROJECT_REPORT.md` — long-form background (optional).

17. Learning Components

ID	Name	Files	Prerequisites	Related	Difficulty	Why it matters
<code>power_and_wiring</code>	Hardware platform & power	block diagram, sketch header	—	<code>pump_actuation</code>	Easy	Physical context; common-ground rule
<code>ultrasonic_sensing</code>	HC-SR04 time-of-flight	<code>controller loop</code>	—	<code>level_conversion</code> , <code>fault_safety</code>	Easy	Source of all data
<code>level_conversion</code>	Distance → %	<code>distanceToPercent</code>	<code>ultrasonic_sensing</code>	<code>control_law</code>	Easy	Inverted mapping is the classic bug

ID	Name	Files	Prerequisites	Related	Difficulty	Why it matters
control_law	10/90 hysteresis	updatePump	level_conversion	manual_override , fault_safety	Medium	Core design idea
manual_override	Debounce override	overridePressed , loop	control_law	control_law	Medium	Priority vs safety limit
fault_safety	Fail-safe design	setup , loop , updatePump	ultrasonic_sensing , control_law	all	Medium	Layered safety, including the physical layer
pump_actuation	L298N drive	applyPumpOutput	power_and_wiring	control_law	Easy	Decision → physical action
lcd_display	Flicker-free LCD	updateDisplay	—	main_loop	Easy	UI and I ² C usage
main_loop	Non-blocking scheduler	setup , loop	all firmware	all	Medium	Ties everything together
logic_tests	Host-side tests	test/logic_test.cpp	control_law , level_conversion	—	Easy	Verifying embedded logic without hardware
legacy_measure_sketch	Original sketch	*_ORIGINAL.ino	ultrasonic_sensing	main_loop	Easy	Shows the design's evolution and bugs avoided

18. Stage 1 Learning Facts

#	Fact	Evidence
F1	The system keeps a tank between 10 % and 90 % full.	README; PUMP_ON_BELOW , PUMP_OFF_ABOVE
F2	The HC-SR04 is mounted above the water and never touches it.	README
F3	HC-SR04 TRIG is on D12 and ECHO is on D10.	controller pins
F4	The LCD is a 16×2 I ² C display at address 0x27 on A4/A5.	lcd(0x27,16,2) , header
F5	The L298N is driven by IN1=D7, IN2=D8 and ENA=D6.	controller pins
F6	The override button is on D2 with INPUT_PULLUP ; LOW means pressed.	setup , overridePressed
F7	Distance is the median of 5 pings (ping_median).	loop , PING_SAMPLES
F8	NewPing returns 0 when no echo is received; the firmware treats 0 as "no reading", never as a distance.	loop comment
F9	A larger distance means a lower level (inverted mapping).	distanceToPercent
F10	SENSOR_TO_EMPTY = 14 cm and SENSOR_TO_FULL = 3 cm are defaults that must be calibrated.	constants comment
F11	The level percentage is clamped to 0–100 and rounded.	distanceToPercent
F12	The pump turns ON when level < 10 %.	updatePump

#	Fact	Evidence
F13	The pump turns OFF when level $\geq$ 90 %.	updatePump
F14	Between 10 % and 89 % the pump keeps its previous state (hysteresis).	updatePump
F15	Hysteresis prevents chatter caused by surface ripple.	README, updatePump comment
F16	pumpRunning is the memory that makes hysteresis possible.	state comment
F17	The 90 % cut-off is checked before the override.	updatePump order
F18	The override is cancelled automatically when the tank reaches 90 %.	updatePump
F19	A sensor fault forces the pump OFF even if the override is on.	updatePump ; test
F20	Five consecutive zero readings set sensorFailed .	MAX_BAD_READINGS
F21	One good reading clears the sensor fault.	loop
F22	The pump is set OFF first in setup() .	setup
F23	Sampling runs every 300 ms using millis() , not delay() .	loop
F24	The button is polled on every loop pass with a 50 ms debounce.	loop , DEBOUNCE_MS
F25	The LCD is redrawn only on change and never cleared, to avoid flicker.	updateDisplay
F26	The serial log runs at 9600 baud: dist=...cm level=...% pump=... .	loop
F27	The L298N and the Arduino must share a ground.	header comment
F28	A 12 V adapter feeds the pump rail through a rocker switch.	block diagram
F29	Overflow water is routed away from the electronics as a physical fail-safe.	README
F30	The current controller is a rewrite; the original was lost.	README
F31	The original sketch only displayed readings under 14 cm and had no pump logic.	*_ORIGINAL.ino
F32	test/logic_test.cpp duplicates the logic and runs 16 checks on a PC.	test file
F33	The tests do not exercise the sensor, LCD or L298N.	test/README

19. Stage 2 Candidates

control_law

- **Why:** state-dependent behaviour and priority order.
- **Files:** updatePump ; tests.
- **Behaviours:** hold band, cancelling the override, fault precedence.
- **Prediction:** given a sequence of levels and an override state $\rightarrow$ the pump state.
- **Failure:** what happens with a single 50 % setpoint, or if the override is checked before the 90 % cut-off?
- **Modification:** change the thresholds; add a minimum on-time; add a maximum run-time.

fault_safety

- **Why:** layered safety and recovery semantics.
- **Files:** setup , loop , updatePump .
- **Behaviours:** the dropout counter, fault latch and clear, safe boot, override persistence (P7).
- **Prediction:** a 4-zero burst vs a 5-zero burst; time to detect a fault.
- **Failure:** treating 0 as distance 0; an unplugged sensor in the legacy sketch.
- **Modification:** clear the override on fault; add a pump run-time watchdog.

level_conversion

- **Why:** inversion, clamping and resolution.
- **Files:** distanceToPercent .
- **Behaviours:** span guard, rounding, about 9 % quantisation (P5).
- **Prediction:** the % for a given cm.

- **Failure:** swapped constants; a non-inverted formula.
- **Modification:** float distance from `µs`; a different tank height.

main_loop

- **Why:** timing, blocking and ordering.
- **Files:** `loop`, `overridePressed`.
- **Behaviours:** millis gating, early returns, blocking inside `ping_median`.
- **Prediction:** order of side effects per cycle.
- **Failure:** replacing millis with `delay(300)`; a missed tap (P8).
- **Modification:** move to `ping_timer`; button interrupt.

lcd_display

- **Why:** caching and format width.
- **Files:** `updateDisplay`.
- **Behaviours:** the static cache, `%-16s` padding, truncation (P6).
- **Prediction:** the exact LCD text for a given state.
- **Failure:** using `lcd.clear()`; no padding ("450%").
- **Modification:** shorten the tag; add a fault code.

20. Ambiguities and Verification Required

Issue	Sources	Confidence	Check
Rewrite never run on hardware	README	DOCUMENTED	Flash and observe
Geometry 14/3 cm not measured	controller comment	DOCUMENTED	Calibrate on the tank
README pin map omits IN2→D8	README vs controller	Omission (README now updated)	Wire IN2 to D8 or tie it LOW
Library: deck says <code>HCSR04.h</code> ; source uses <code>NewPing</code>	<code>seminar_deck.pptx</code> vs controller	CONFLICT — historical (deck describes the 2024 build)	Which library the lost original used is UNKNOWN
Accuracy "about ±0.5 cm"	README	DOCUMENTED estimate; not characterised. The rewrite resolves only 1 cm.	Measure
How the Arduino is powered (VIN, barrel jack or L298N 5 V)	Diagram shows only switch → Arduino	NOT VERIFIED	Inspect the build
Overflow plumbing	README only	DOCUMENTED; not verifiable from the repo	Inspect the tank
Tank volume about 5 L	Diagram label	DOCUMENTED	—
Purpose of <code>src/water_level_measure.ino</code> (identical to ORIGINAL)	md5 match	VERIFIED duplicate; purpose UNKNOWN	Ask the author
<code>docs/images/readme</code> (2-byte placeholder file)	git	VERIFIED; no content	—
Team member name: "Mohammed Ruwaif" (README) vs "Ruwaif Fakardey" (deck)	README, deck	CONFLICT — REQUIRES VERIFICATION	Ask the author
Library versions	none	UNKNOWN	Pin in the README if reproducibility matters

Current Project Snapshot

- **Last inspected:** 2026-10-01
- **Branch / commit:** `main` @ `8f3b88634aa2dff892dfdfa7ac0ec1945bd72991` (2026-08-24, "Rename docs/test/README.md to test/README.md")
- **History:** 10 commits (2026-07-19 → 2026-08-24), all uploaded through the GitHub web UI. No tags or releases.

- **Recently changed (git):** `test/README.md`, `test/logic_test.cpp` (moved from `docs/test/`), `docs/images/*`
- **Untracked (added 2026-09-27 → 2026-10-01):** `docs/report/PROJECT_REPORT.md/.pdf`, `docs/images/hysteresis_cycle.svg`, `CODEMAP.md`, `CODEMAP.pdf`. `README.md` was modified.
- **Sensitive files:** none found.

21. Project Snapshot

```
PROJECT: Ultrasonic Water-Level Sensor & Controller
TYPE: Embedded firmware + hardware prototype (Arduino), academic mini-project
LANGUAGES: Arduino C++, C++
FRAMEWORKS: Arduino core (AVR); libs NewPing, Wire, LiquidCrystal_I2C
MAIN_COMPONENTS: ultrasonic_sensing, level_conversion, controlLaw, manual_override, fault_safety, pump_actuation,
lcd_display, main_loop, logic_tests, legacy_measure_sketch, power_and_wiring
ENTRY_POINTS: src/water_level_controller.ino (setup/loop); test/logic_test.cpp (main)
MAIN_WORKFLOWS: boot-safe-off; 300ms sense→convert→decide→actuate→display→log; sensor-fault latch/clear; manual top-up;
calibration
HARDWARE: Arduino Uno, HC-SR04, 16x2 I2C LCD 0x27, L298N, 12V submersible pump, button, rocker switch, 12V adapter
EXTERNAL_SERVICES: none
DATABASE: none
TESTING: 16 host-side logic checks (g++); hardware untested for rewrite
STATUS: demonstrated on hardware (original fw lost); rewrite logic-verified, awaiting hardware re-test
```